Federated O-RAN Slice SLA Prediction: A Cross-Architecture Empirical Benchmark on Colosseum/ColO-RAN

Hao-Chun Tsai, *Student Member, IEEE*

***Abstract*—Federated learning (FL) on O-RAN edge gNBs is the natural paradigm for slice-level SLA-violation forecasting because per-user telemetry cannot be centrally pooled. Despite a decade of FL benchmark research, the deployment design space — sequence model architecture, federation algorithm, client-heterogeneity partition, commodity edge hardware — has been studied one axis at a time, leaving the joint behaviour under realistic O-RAN traffic characterised only anecdotally. This paper presents the first cross-architecture empirical FL benchmark on the publicly-available Colosseum/ColO-RAN testbed: 3 sequence backbones (LSTM, Mamba, Spiking-SSM) $\times$ 5 algorithms (FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn) $\times$ 6 partitions (natural-by-BS $+$ Dirichlet $\alpha \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$) $\times$ 10 seeds $=$ 900 cells, with NVML idle-baseline-subtracted training-energy measurement on RTX 4080 commodity hardware, augmented by a 15-cell controlled random-split ablation on $4\times$ Tesla V100-SXM2-32GB to probe the leading mechanism hypothesis. We report four empirical findings: (1) client heterogeneity in O-RAN is structurally helpful, not harmful — the natural base-station partition uniformly outperforms every parametric Dirichlet α across all 3 architectures $\times$ 5 algorithms, inverting the standard FL-benchmark assumption; (2) the algorithm-design space is flat at FedAdam-saturated headroom ($\leq +0.016$ AUC over FedAvg), bounding the room available to recent sharpness-aware methods; (3) selective-state-space backbones interact catastrophically with control-variate algorithms at moderate partition skew (Mamba $\times$ SCAFFOLD $\times$ $\alpha \in \{0.10, 0.50\}$); (4) architecture choice traverses $10\times$ in commodity-GPU energy and an order-of-magnitude larger AUC range than algorithm choice, dwarfing the algorithmic-leverage scale. We provide a reference benchmark with paired-bootstrap CI_{95} over 270 paired distributions, NVML-instrumented per-cell energy, and pre-registered statistical protocol; the artefact is released under Apache-2.0 with Croissant dataset metadata and a Zenodo DOI on paper acceptance.**

***Index Terms*—Federated learning, O-RAN, slice management, SLA prediction, state-space models, spiking neural networks, NVML energy measurement, paired-bootstrap inference.**

I. Introduction

OPEN Radio Access Networks (O-RAN) decomposes the cellular base station into commodity hardware components controlled by ML-driven xApps and rApps running on disaggregated RAN Intelligent Controllers (RICs) [1]. Slice-level Service Level Agreement (SLA) violation forecasting is one near-RT RIC xApp pattern of practical interest: each gNB simultaneously serves heterogeneous traffic slices (the ColO-RAN release exercises eMBB constant-bitrate, MTC Poisson, and URLLC Poisson — see Section III), and proactive forecasting of next-second SLA risk informs near-RT RIC resource-allocation decisions within its 10 ms – 1 s control loop. Because edge gNBs accumulate per-user telemetry that cannot be centrally pooled, federated learning (FL) is the natural training paradigm: each gNB trains a local sequence model on its own traces and shares only weight updates with the FL aggregator. We place the aggregator in the non-RT RIC / SMO orchestration layer ($\geq$ 1 s timescale fits our 100-round federated training cadence per Section IV); a near-RT-RIC-internal xApp aggregator would alternatively be possible if cross-cell xApp messaging supports the federation protocol — we do not engage with that deployment alternative beyond noting it.

Standard FL practice — codified by a decade of toy-image benchmarks — treats client heterogeneity as a *wound* to be healed: lower Dirichlet α (more skew) drives lower accuracy, motivating sharpness-aware methods, control variates, and proximal regularizers. **In O-RAN slice SLA prediction, this premise inverts.** When clients are the *natural* base stations of a Colosseum/ColO-RAN testbed (one client per gNB), the federated average across 7 base-station-natural clients consistently outperforms every uniform-Dirichlet partition of the same training data on the same model + algorithm. Empirically (Section **??**, all values are out-of-sample test AUC averaged over 10 seeds), the natural-by-BS partition achieves AUC 0.913–0.918 on LSTM (5-algo range; $\Delta \approx 0.005$), 0.8998–0.9186 on Mamba (with SCAFFOLD as the lower outlier and the other four algorithms in 0.911–0.919), and 0.840–0.856 on Spiking-SSM, **strictly above the AUC obtained at any parametric Dirichlet** α for every (arch, algo) pair, and monotonically increasing as $\alpha \to 0$ within the Dirichlet sweep. The $\alpha = 5.0 \to \alpha = 0.05$ AUC gap is $\approx$ 0.10–0.12 on the dense LSTM/Mamba backbones; on Spiking-SSM the gap is smaller (≈ 0.022) but still positive, and across all five algorithms heterogeneity is associated with higher AUC, not lower. The mechanism is open — natural-by-BS is *not* the low-α limit of the Dirichlet sweep (the two partition families redistribute rows along orthogonal axes, see Section IV-C) — and we present a controlled `random_split` ablation in Section **??** (15 cells $\times$ 3 architectures on $4\times$ Tesla V100-SXM2-32GB) showing that any partition that breaks bs grouping (whether by per-slice Dirichlet redistribution at $\alpha = 5.00$ or by uniform random shuffling) collapses AUC to a tight band ~ 0.18 below natural-by-BS, and is statistically equivalent to Dirichlet $\alpha = 5.00$ within seed σ. The mechanism

H.-C. Tsai is with the Department of Computer Science, National Yang Ming Chiao Tung University, Hsinchu, Taiwan. E-mail: hctsai1006@cs.nctu.edu.tw. ORCID: 0009-0009-7115-0149.

direction (preservation of bs grouping) is therefore confirmed; the precise dataset axis the model exploits via bs grouping (channel-state features, see Section **??**) remains under further investigation.

Despite the explosion of FL benchmarks since 2018, the deployment design space for FL-based slice SLA prediction remains under-characterized. The space spans four axes:

1) **Sequence model architecture** — recurrent (LSTM), structured state-space (Mamba), or bio-inspired sparse spiking variants (Spiking-SSM).
2) **Federation algorithm** — FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn (and the more recent sharpness-aware family discussed in Section II).
3) **Client heterogeneity** — including the natural base-station partition (one gNB $\Rightarrow$ one client) and parametric Dirichlet stress $\alpha \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$ over slice IDs.
4) **Commodity hardware** — consumer-tier edge GPUs (RTX-class) at the gNB rather than data-center accelerators.

Existing benchmarks address each axis in isolation. NeurIPS-class FL benchmarks evaluate algorithm $\times$ heterogeneity on toy image data [2], [3], [4], [5]. Spiking-NN benchmarks compare architectures in centralized regimes on neuromorphic-friendly datasets [6]. FL $\times$ wireless papers focus on a single architecture [7], [8], [9]. The implicit assumption underlying single-axis recommendations: **best architecture $\times$ best algorithm $\times$ representative α $\times$ commodity hardware = best deployment**.

This paper tests the assumption empirically. We construct the first comprehensive 4-axis FL benchmark on the publicly-available Colosseum/ColO-RAN testbed dataset [1], [10], sweeping {LSTM, Mamba, Spiking-SSM} $\times$ {FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn} $\times$ {natural-by-BS, Dirichlet $\alpha \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$} $\times$ 10 seeds = 900 cells (90 groups, all 10 seeds populated), with NVML-instrumented per-round GPU energy measurement on RTX 4080 (sm_89) commodity hardware. Statistical inference uses paired-bootstrap CI_{95} on per-seed AUC deltas; Wilcoxon p-values are reported uncorrected per finding (with family-wise Bonferroni discussed in Section IV-E as a supplementary check).

Contributions:

1) **Inverted heterogeneity in O-RAN slice federation**: against the standard FL-benchmark assumption, the natural base-station partition uniformly outperforms every parametric Dirichlet α in our sweep across all 3 architectures and 5 algorithms. We do not claim a closed-form mechanism — Section **??** reports a controlled `random_split` ablation that tests the leading "bs-conditioned signal preservation" hypothesis — but the empirical pattern is the first published characterization on Colosseum/ColO-RAN and is deployment-relevant regardless of which mechanism candidate ultimately survives.
2) **Algorithm-design space is flat in this regime; FedAdam achieves the empirical maximum in our 5-algorithm baseline**: across Dirichlet $\alpha \in \{0.05..5.0\}$, the spread between the best and worst of {FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn} on a fixed (arch, partition) cell is 0.022–0.031 AUC on LSTM/Spiking and 0.078 on Mamba (driven by a single SCAFFOLD outlier). FedAdam consistently leads FedAvg via paired-bootstrap CI_{95} (LSTM Dirichlet-averaged $\Delta = +0.0080$, range $+0.0048.. + 0.0128$, all 5 cells significant; Mamba Dirichlet-averaged $\Delta = +0.0062$, range $-0.0021.. + 0.0097$, 4 of 5 cells significant; Spiking Dirichlet-averaged $\Delta = +0.0164$, range $+0.0108.. + 0.0267$, 4 of 5 cells significant). The mechanism analysis in Sections II-F + **??** + **??** argues that LookAhead-style server-side aggregation (FedSWA / FedSCAM) is *unlikely to systematically exceed* this empirical maximum because of the variance-estimation gap between Adam-style and LookAhead-style server steps (predictive claim, not theorem); we accordingly dismiss FedSWA [11] and FedSCAM [12] via mechanism analysis rather than benchmark them. We do not benchmark FedBN [13] either; Section **??** L2 discusses why the algorithm-flatness ranking is unlikely to be overturned by FedBN given its reported gain envelope on related cellular feature-skew tasks.
3) **Architecture $\times$ algorithm catastrophic interaction (Mamba $\times$ SCAFFOLD $\times$ $\alpha \in \{0.10, 0.50\}$)**: SCAFFOLD on Mamba at $\alpha = 0.10$ yields AUC 0.7609 ± 0.0830 — std amplified $\sim 3\times$ over (Mamba, FedAvg) at the same α (0.0251) and $\sim 10\times$ over (Mamba, FedAvg) at $\alpha = 5.0$ (0.0073). The deployment warning is concrete: control-variate methods interact pathologically with selective-scan SSMs under high heterogeneity, even as both components individually behave well on neighboring cells.
4) **Architecture leverage dominates algorithm leverage** (paired-bootstrap CI_{95}) **on RTX 4080**: on the energy-AUC Pareto front (Section **??**), architecture choice spans $10\times$ in NVML-measured model-attributable energy on RTX 4080 (LSTM $\sim$ 3.5 kJ $\rightarrow$ Mamba $\sim$ 10.5 kJ $\rightarrow$ Spiking $\sim$ 41 kJ per cell). Holding (algorithm, IID partition) fixed, the paired Spiking $-$ LSTM AUC delta is concentrated at -0.061 to -0.074 across all 5 algorithms (all 5 cells significant at $p < 0.005$), substantively dwarfing the FedAdam $-$ FedAvg algorithmic gap of $+0.006$ to $+0.016$ on the same architectures (contribution 2). The $10\times$ span is hardware-specific: the same architectures on a sparsity-aware accelerator (Loihi-2, IBM NorthPole) would yield a substantially smaller LSTM-vs-Spiking energy ratio (Section **??** + Section **??** L1, L6). Operator decisions on commodity edge GPU should center architecture-energy trade-off, not algorithm breadth.
5) **Open reproducible reference benchmark**: spec-driven YAML pipeline with pre-flight algorithm validation and `--skip-completed` resumability; paired-bootstrap statistical pipeline; NVML idle-baseline-subtracted training energy; Croissant metadata; Zenodo DOI on acceptance; Apache-2.0 license (preregistered). The full 4-axis table

maps deployment configurations → (AUC, F1, energy, paired CI_{95}) over 90 (arch × algorithm × partition) groups, enabling lookup-style operator decisions on Colosseum/ColO-RAN.

The remainder of the paper is organized as follows. Section II reviews related work across the four axes. Section III describes the Colosseum/ColO-RAN dataset and our preprocessing pipeline. Section IV details the architecture, algorithm, and partition methodology. Section V enumerates the reproducibility infrastructure and our Croissant-aligned data release. Section **??** reports results across the 90 (arch × algorithm × partition) groups with paired-bootstrap CI_{95}. Section **??** discusses the mechanism finding and articulates the applicability boundary on commodity edge hardware. Section **??** enumerates limitations. Section **??** concludes.

II. Related Work

A. Federated learning benchmarks

The canonical FL benchmark is **LEAF** [2], which standardized federated MNIST/Shakespeare evaluation pipelines for early FedAvg variants. **Flower** [14] later provided a framework rather than a benchmark, supporting heterogeneous device experiments. **FedScale** [3] scaled to thousands of cross-device clients; **FLAIR** [4] introduced a long-tail multi-label image dataset. **pfl-research** [5] integrates differential-privacy mechanisms into a simulation framework. Most recently, **fev-bench** [15] introduced rigorous bootstrap-CI_{95} evaluation for time-series forecasting, though without the federated dimension. None of these benchmarks targets O-RAN slice SLA prediction specifically; their toy-data settings cannot capture the covariate-skew structure of real RAN telemetry.

B. FL on cellular and wireless networks

A first generation of FL × cellular work targeted SLA-constrained slicing under the Beyond-5G banner. **Statistical FL for B5G SLA-Constrained RAN Slicing** [16] introduced a long-term CDF SLA constraint. **CDF-Aware FL** [17] formalized the constraint as a per-client penalty. **Fed-LSTM** [7] deployed an LSTM forecaster for slice-level traffic. **TRACTOR** [18] integrated reinforcement-learning xApps for resource-block assignment. **REAL** [19] integrated the OSC near-RT RIC with srsRAN for RL-based slice-resource-allocation xApps under urban-mobility channel modelling. Recently **Hayek et al. 2025** [20] measured FL convergence on a 5G/WiFi/Ethernet COTS testbed with Raspberry-Pi clients, exposing uplink straggler effects. **arXiv:2508.08479** [8] benchmarked FedAvg/FedProx/FedBN on five throughput-prediction datasets with LSTM, CNN, CNN+LSTM, and Transformer architectures, finding FedBN superior under feature skew. **Mangi et al. 2026** [9] ("When Clients Drift") proposed regime-aware aggregation under leave-one-regime-out evaluation on synthetic 6G RAN telemetry.

Each of these papers covers one or two of our four axes. None benchmarks LSTM, Mamba, and Spiking-SSM jointly under parametric Dirichlet heterogeneity, and none reports per-round NVML energy. Our work is differentiated by the combination of architecture breadth, algorithm breadth, parametric heterogeneity, and energy instrumentation on the **publicly-available** Colosseum/ColO-RAN dataset, in contrast to closed simulators or undisclosed datasets used by Mangi et al.

C. Spiking-SSM and FL × spiking neural networks

Hybrid spiking-state-space architectures emerged from late 2024. **SpikMamba** [21] combined LIF spiking neurons with Mamba selective scan for event-camera action recognition. **Mamba-Spike** [22] introduced a spiking front-end for general temporal data. **Spiking Point Mamba** [23] extended to 3D point clouds. **SpikingSSMs** [24] formalized sparse-parallel spiking state-space layers. **SpikingMamba** [25] distilled large language models into spiking variants for energy efficiency. **SpikingBrain** [26] scaled spiking architectures to 76B parameters on data-center GPU clusters with explicit FLOP-utilization measurement.

Federated-learning variants of spiking neural networks emerged earlier than spiking-SSM: **arXiv:2106.06579** [27] founded FL × SNN. **FedLEC** [28] extended to label-skew non-IID. **SNN in vertical FL** [29] examined VFL energy trade-offs. **Robustness of SNN in FL with compression** [30] addressed Byzantine attacks. **Privacy in FL with SNN** [31] characterized gradient leakage. Most recently, **arXiv:2602.12009** [32] examined firing-rate sensitivity to differential privacy in federated SNN — preempting the DP-SNN-FL angle. We accordingly cite this work in Section **??** as a deferral and do not duplicate the DP study; our contribution is orthogonal in its multi-architecture × parametric-Dirichlet × NVML scope.

To our knowledge, no published work combines spiking-SSM with FL on cellular/RAN telemetry. Our `spiking_expand2` variant adapts the wide-inner-state design pattern from Mamba [33] to spiking-SSM and demonstrates competitive AUC/energy trade-off on commodity edge GPU under FL.

D. GPU energy measurement methodology

The argument that FLOP counts mis-estimate actual hardware energy predates our work. **Shen et al. 2023** [34] ("Is Conventional SNN Really Efficient?") critiqued synaptic-operation accounting via a Bit Budget framework, finding that quantized ANNs dominate SNNs at equivalent bit budgets. **α-FLOPs** [35] proposed a parallelism-aware alternative. **NeuroBench** [36] standardized neuromorphic benchmarking on real hardware. The **ML.ENERGY Benchmark** [37] measured inference energy on generative AI; **Where Do the Joules Go?** [38] diagnosed the breakdown across model families. **STEP** [6] specifically benchmarked spiking transformers including memory-access cost. **Beyond Backpropagation** [39] applied NVML and CodeCarbon to alternative training objectives.

We follow the **Energy API** approach (`nvmlDeviceGetTotalEnergyConsumption`, available on Volta+ with NVML $\geq$ 11.0) preferred over polling-power Riemann-sum integration per the ML.ENERGY 2025 paper [37] and accompanying leaderboard. Our novel angle is the **federated-training regime**: prior NVML benchmarks

measure inference [36], [37] or centralized training [26], [39]; we measure per-round energy across an FL sweep, including idle-baseline subtraction.

E. Heterogeneity in federated learning

NIID-Bench [40] established Dirichlet partitioning as the standard non-IID benchmark. **FedBN** [13] keeps client-local BatchNorm statistics for feature-skew settings; **arXiv:2508.08479** [8] demonstrated FedBN's superiority on cellular feature-skew. **pFedFDA** [41] specifically targets covariate shift. **Borazjani et al. 2025** [42] argued for embedding-based heterogeneity definitions beyond label-skew.

We use Dirichlet partitioning over slice IDs as the canonical covariate-skew model for O-RAN: each gNB serves a different mix of slice traffic, with mixing entropy controlled by α_{dir}. We adopt $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$ to span extreme stress (each gNB dominated by a single slice) through near-IID. We additionally evaluate the natural base-station partition (one gNB $\Rightarrow$ one client, 7 ColO-RAN gNBs in the public release) as a non-parametric heterogeneity reference. We discuss the applicability of embedding-based alternatives in Section **??**.

F. Sharpness-aware federated learning (2024–2026)

A separate strand of recent FL work targets *heterogeneity-induced sharp minima* via per-step learning-rate cycling and server-side weight extrapolation. **FedSAM** [43] adapted Foret et al.'s SAM perturbation to the federated client step. **FedSWA** [11] introduced an intra-round cyclical local LR schedule (eq. 3) combined with a LookAhead-style server EMA $\theta_t = \theta_{t-1} + \alpha(v_t - \theta_{t-1})$ with $\alpha = 1.5$ (eq. 17). **FedSCAM** [12] composed SAM with clustering for further sharpness regularization. The reported gains (FedSWA: $+5.6\%$ over FedAvg on CIFAR-100 with 10% client participation across 1000 rounds) assume large-N low-participation FL with coherent client drift toward sharp minima.

We do not empirically evaluate this family on Colosseum/ColO-RAN (see Section **??**). Two structural mismatches make the absence of comparison defensible on mechanism alone. First, our regime — 7 base-station-natural clients with 71% per-round participation, 100 rounds, RAN slice SLA prediction — is *aggregation-noise-dominated* rather than coherent-drift-dominated. Second, FedAdam's adaptive variance damping $\mathrm{Adam}(m, v)$ over $(v_t - \theta_{t-1})$ systematically dominates FedSWA's variance-blind LookAhead overshoot *in expectation* in this regime: FedAdam knows when to step small (high v) and when to step normally (low v), while FedSWA's LookAhead extrapolation rate α_{LA} (the FedSWA paper's α-symbol; we write it α_{LA} throughout to disambiguate from the Dirichlet α_{dir} of Section IV-C and the FedDyn α_{dyn} of Section IV; FedSWA's preregistered value is $\alpha_{\text{LA}} = 1.5$) amplifies *both* signal and noise uniformly. Empirically (Section **??**), FedAdam already saturates the headroom available to server-side aggregation modifications at $+0.006$–$+0.016$ Dirichlet-averaged AUC over FedAvg across the 3 architectures (paired-bootstrap CI_{95} in Section I contribution 2); LookAhead overshoot is unlikely to *systematically* exceed this ceiling without the variance-estimation machinery FedAdam has and FedSWA lacks. We additionally note that FedSWA targets *heterogeneity-as-sharpness-wound*, which our inverted-α finding (Section I, Section **??**) indicates does not characterize this dataset.

TABLE I
COLO-RAN SLICE AND SCHEDULER INDEX MAPPING.

Index	Slice ID	Traffic class	Workload
0	eMBB	broadband	4 Mbps CBR/UE
1	MTC	machine-type	Poisson 30 pkt/s, 125 B/UE
2	URLLC	ultra-reliable	Poisson 10 pkt/s, 125 B/UE
Index	**`sched` ID**	**Scheduler**	
0	RR	Round-Robin	
1	WF	Waterfilling	
2	PF	Proportionally Fair	

III. DATASET

A. Colosseum / ColO-RAN testbed

ColO-RAN [1] is the publicly-released RAN telemetry corpus collected from the Colosseum digital-twin testbed [10], comprising the per-second downlink/uplink physical-layer measurements of 7 base stations (Colosseum Rome scenario, BS nodes $\{1, 8, 15, 22, 29, 36, 43\}$; we re-index these to `bs_id` $\in \{1, \ldots, 7\}$ for ease of use) operating concurrently under 28 distinct traffic configurations (`tr` $\in \{0, \ldots, 27\}$, each a different RBG-per-slice allocation pattern; the exact (slice 0 / slice 1 / slice 2) RBG triples per `tr` are documented in the ColO-RAN release). Each base station serves 3 logical slices, and each slice can be served by one of 3 scheduling policies; both index mappings are listed in Table I. The unified telemetry table contains ≈ 18 M rows of (timestamp, `bs_id`, `slice_id`, `sched`, `tr`) keyed observations with 17 continuous features (uplink/downlink throughput, MCS/CQI, BLER, RB utilisation, buffer occupancy; full list in Section IV). We use the public release without modification; no synthetic augmentation, no re-simulation.

B. Task definition: per-slice SLA-violation forecasting

The forecasting target is the binary indicator $y_{t+1} = \mathbf{1}[ul\_bler_{t+1} > 0.10]$, predicting whether the next-second uplink BLER on a given (`bs_id`, `slice_id`) tuple will breach the 10% SLA threshold. This threshold is the canonical SLA-violation gate used by the ColO-RAN authors [1]; the binary framing keeps the metric (AUC, F1) decision-relevant for the near-RT RIC's resource-allocation xApps. The empirical positive rate on the train partition is 30.9% (per-slice 34.9% / 26.7% / 31.0%, measured on `tr` $\in \{0..21\}$; see Section **??** for how this rate enters the per-client loss reweighting). Inputs to the model are the most recent five seconds of (`slice_id`, `sched`, `tr`, 17 continuous features) on the same (`bs_id`, `slice_id`) tuple — `seq_len=5` is preregistered from v3/v4.

C. Out-of-distribution split by traffic config

To prevent label leakage between train / validation / test, we partition the 28 traffic configurations into three disjoint groups:

train `tr` $\in \{0..21\}$ (22 configs), validation `tr` $\in \{22..24\}$ (3 configs), and test `tr` $\in \{25..27\}$ (3 configs). All three sets are evaluated on the same 7 base stations and 3 slices; only the traffic-config index changes. This split is identical to v4 so accuracy numbers are cross-paper comparable. Crucially, **partition-into-clients (Section IV-C) operates within the train `tr` range only**: validation and test sets are pooled across all clients to guarantee identical evaluation surfaces across configurations.

D. Target-leakage audit (preserved from v4)

A v4-era audit established that the previously documented `allocation_efficiency = 0.5 * throughput_eff + 0.3 * qos + 0.2 * prb_util` regression target is a deterministic linear combination of three concurrent inputs and is therefore unsuitable. We retain that exclusion and the v4-validated leak-free continuous feature set in v7.

E. Preprocessing pipeline

The 18 M-row unified parquet (`coloran_raw_unified.parquet`) is materialised once and streamed lazily through four pipeline stages: (i) per-(`bs_id`, `slice_id`) sequence construction with `build_run_sequences` (rolling `seq_len=5` windows, positive-rate computed from train rows only via `pos_weight_split=train` (preregistered) — this avoids test-set leakage into the loss function); (ii) leak-free `ContinuousScaler` fit on the train partition only and applied to validation and test; (iii) categorical encoding of `slice_id` × `sched` × `tr` via `nn.Embedding` lookups; (iv) Dirichlet-or-natural-by-BS client partition (Section IV-C). All four stages are implemented as pure functions in `src/fl_oran/data_v2/` and tested in `tests/test_v5_*` for determinism and seed-reproducibility (single-source-of-truth contract). The same sequence index is shared across all algorithms within an (arch, partition, seed) triple via a `SharedSplits` cache; this is the perf inheritance from prior centralized work.

IV. Method

A. Three architectures with a shared encoder–classifier shell

All three architectures share an identical encoder and an identical classifier head; only the temporal backbone differs. The encoder concatenates `nn.Embedding`-projected categorical features with the continuous features pre-standardised by the leak-free scaler (Section III), producing a per-step input vector. The classifier head is a two-layer MLP `Linear(b -> 64)-> ReLU -> Linear(64 -> 1)` consuming the backbone's last-step activation, where b is the backbone's exposed dimension. Hyperparameter parity within $\pm 10\%$ of the LSTM baseline parameter count is preregistered and verified by `tests/test_v6_param_count.py` (LSTM 44 553, Mamba 40 489, Spiking-expand2 43 593). The negative direction of the Mamba parity gap (-9% relative to LSTM, vs Spiking-expand2's -2%) reflects Mamba's structurally smaller state-space block at our preregistered configuration ($d_{\text{model}} = 64, d_{\text{state}} = 16, \textit{expand} = 1$); we did not search over Mamba block dimensions to close this gap because (a) the v6 architecture audit pinned these block dimensions and (b) $\pm 10\%$ parity was the preregistered design constraint, not a hyperparameter optimisation outcome. The asymmetry should not be interpreted as compute-budget shortfall: at fixed `seq_len=5` and `max_steps=50`, gradient-step compute is dominated by the linear encoder/classifier shared across architectures, not by the backbone's parameter count.

LSTM baseline (`ForecasterV2`). Two-layer stacked `nn.LSTM` (input $\to$ 64 $\to$ 32). Unchanged since v3. Backbone exposes $b = 32$.

Mamba-S6 (`MambaForecaster`). `Linear(input -> 64)` $\to$ 2× `MambaS6Block` ($d_{\text{model}} = 64$, $d_{\text{state}} = 16$, $d_{\text{conv}} = 4$, $\textit{expand} = 1$) $\to$ `Linear(64 -> 32)`. Each block performs (a) input-dependent projection of $(\Delta t, B, C)$, (b) depthwise causal 1-D convolution on the gating branch with SiLU non-linearity, (c) sequential discretised SSM scan, and (d) gated down-projection — implementing the selective state-space block of Gu and Dao [33]. We use a pure-PyTorch sequential-scan implementation rather than the `mamba-ssm` Triton kernel so the reproducibility artefact requires only a PyTorch + CUDA runtime, not a system CUDA-dev toolkit. Backbone exposes $b = 32$.

Spiking-SSM (`spiking_expand2`). `Linear(input -> 56)` $\to$ 2× `SpikingSSMBlock` ($d_{\text{model}} = 56$, $d_{\text{state}} = 16$, $\textit{expand} = 2 \Rightarrow d_{\text{inner}} = 112$, `lif_threshold=1.0`, `lif_beta=0.9`, `atan_alpha=2.0`) $\to$ `Linear(56 -> 32)`. Each block performs (a) input projection, (b) diagonal SSM recurrence with learnable B, C, D matrices and log-parameterised A initialised at $-[1, 2, \ldots, d_{\text{state}}]$ per channel, (c) per-channel `snntorch.Leaky` LIF neuron emitting binary spikes via the atan surrogate gradient [44], (d) `out_proj` linear consuming the binary spike train. The $\textit{expand} = 2$ widening (vs $\textit{expand} = 1$ for the v6 default) is the architectural variant identified during prior wide-state experiments — it dominates the alternative wide-Mamba (`mamba_expand2`) variant on energy-AUC trade-off (-11% energy at parity AUC vs $+12\%$ theoretical / $+4\%$ measured energy at no AUC gain). Backbone exposes $b = 32$.

B. Five federated-learning algorithms

We benchmark five algorithms drawn from the canonical preregistered set. In what follows we use α_{dir} for the Dirichlet partition concentration parameter (Section IV-C) and α_{dyn} for the FedDyn regularisation coefficient, to disambiguate two distinct α's that appear in the FL literature with the same Greek letter.

FedAvg [45] — server-side aggregate is the per-client `num_examples`-weighted average of client final states (`scripts/aggregation.py`: `weighted_average_state_dicts`). In our spec each client trains for `max_steps_per_round=50` × `batch_size=64` = 3200 examples per round, so `num_examples` is constant across clients and the weighted average reduces to a uniform mean — but we mention the implementation detail explicitly because the released artefact aggregates by `num_examples` and a different spec (varying `max_steps` per client) would no longer give uniform weighting.

FedProx [46] — FedAvg client step augmented with the canonical proximal regulariser, eq. (1), with $\mu = 0.01$.

$$\mathcal{L}^{(i)}_{\text{prox}} = \mathcal{L}^{(i)} + \frac{\mu}{2} \left\| w - w_g \right\|^2 \tag{1}$$

We inject the equivalent $\mu \cdot (w - w_g)$ correction directly into `p.grad` after `loss.backward()` (this is mathematically equivalent to adding the prox term to the loss but avoids building autograd nodes over every parameter).

FedAdam [47] — server-side Adam over the FedAvg pseudogradient $(v_t - \theta_{t-1})$ with first/second-moment coefficients $\beta_1 = 0.9$, $\beta_2 = 0.99$ (note: $\beta_2 = 0.99$ follows the FedAdam paper's choice rather than the canonical Adam default 0.999) and server learning rate $\eta_{\text{server}} = 0.01$.

SCAFFOLD [48] — FedAvg client step augmented with a per-client control-variate correction $+(c_{\text{global}} - c_{\text{local},i})$ applied to gradients post-backward via the `train_one_client_capped(grad_hook=...)` extension point introduced in our prior algorithm-interface design. For the per-client control-variate update we adopt the Adam-friendly Option-II variant as the default; eq. (2) lists both options:

$$c_{\text{local},i} \leftarrow \begin{cases} \nabla \mathcal{L}_i(w_g) & \text{(Option-II, default)} \\ \dfrac{w_g - w_l}{K\,\eta} & \text{(Option-I, canonical SGD)} \end{cases} \tag{2}$$

Option-I implicitly assumes SGD dynamics where weight drift scales with $grad \cdot \eta$; under our Adam local optimiser the drift is $\sim \eta$ regardless of gradient magnitude, making the Option-I c_i estimate $\sim 100\times$ the true gradient magnitude and unstable across rounds.

FedDyn [49] — we use the Adam-friendly Option-II variant as the default (with $\alpha_{\text{dyn}} = 0.01$); the canonical SGD-friendly variant diverges to NaN under our Adam local optimiser at the 100-round budget (root-cause analysis: positive-feedback between Adam's adaptive scaling and the unbounded h-accumulator growth). Both options are listed in eq. (3); the canonical variant is preserved as an opt-in `update_mode="canonical"` for downstream researchers running SGD-only client optimisers.

$$h_i \leftarrow \begin{cases} h_i - \alpha_{\text{dyn}} \cdot \nabla \mathcal{L}_i(w_t) & \text{(Option-II, default)} \\ h_i + (w_l - w_g) & \text{(canonical SGD)} \end{cases} \tag{3}$$

MOON [50] is deferred (preregistered before evaluation): its `forecaster_encode_fn` is hard-bound to the LSTM backbone in v5 and the contrastive-feature extraction does not generalise cleanly to selective-scan SSM or to spiking-SSM activations; we leave the cross-architecture extension to future work and exclude MOON from the headline comparison rather than benchmark it on LSTM-only.

The five algorithms share a common `FLAlgorithm` protocol with explicit `init_state` / `local_train` / `server_aggregate` methods; each algorithm-specific contribution is injected as a closure over the shared `train_one_client_capped` function, never as a duplicate training loop. The single deviation from the v5 $\rightarrow$ v7 design is one optional grad-hook parameter on `train_one_client_capped` for the SCAFFOLD control-variate correction.

C. Partition schemes: natural-by-BS and parametric Dirichlet

We evaluate two partition families holding 7 clients fixed.

Natural-by-BS. One client per ColO-RAN base station (`bs_id` $\in \{1, \ldots, 7\}$). Each client receives its own gNB's full slice mix, scheduler distribution, and traffic-config samples (within the train `tr` range). This is the partition that arises naturally in deployment, and is the central object of contribution 1 (Section I). *Code-name caveat:* in our codebase this partition is implemented as `partition_clients(mode="iid")`. The string `"iid"` is a misnomer inherited from earlier development before we measured the per-bs distributions; the partition is *not* statistically IID across clients on this dataset (per-bs continuous-feature distributions differ measurably — see Section **??**). We keep the legacy `mode="iid"` API name for backwards compatibility with the v5 / v6 codebase but refer to the partition as "natural-by-BS" throughout the paper. Researchers reproducing our pipeline should expect the file structure `artifacts/v7_stage2_full/v7_<arch>_<algo>_iid_n7_s<seed>/` to correspond to natural-by-BS cells.

Parametric Dirichlet. For each value of $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$, we draw per-client slice-id mixing proportions from $\text{Dirichlet}([\alpha_{\text{dir}}, \alpha_{\text{dir}}, \alpha_{\text{dir}}])$ (3-dim, one component per `slice_id` $\in \{0, 1, 2\}$) and assign train rows to clients according to those proportions. $\alpha_{\text{dir}} \rightarrow 0$ concentrates each client on a single dominant slice (extreme covariate skew); $\alpha_{\text{dir}} \rightarrow \infty$ approaches stratified uniform. This follows the canonical Hsu et al. construction [51] and is cross-comparable with NIID-Bench [40]. Implementation: `src/fl_oran/data_v2/partition.py partition_clients(mode="dirichlet")`.

For every partition we use 7 clients, 100 rounds, 50 max steps per round, batch size 64, bf16 mixed precision, and `cudnn_deterministic=True`. The per-round client sample is 5 of 7 (71% participation). Architecture-specific hyperparameter overrides (`lr=5e-4` across all three; `lr_warmup_rounds=5` for Spiking-SSM, 3 for the others) are preregistered in `experiments/specs/stage2_full.yaml`.

D. NVML training-energy measurement

Per-round training energy is measured via `nvmlDeviceGetTotalEnergyConsumption` (NVIDIA Energy API, available on Volta+ architectures) bracketed by `torch.cuda.synchronize()` at round boundaries. We record both the raw cumulative energy and an **idle-baseline subtraction**: a 2-second NVML sample is taken before training to estimate the GPU's idle power draw P_{idle} (mW), and per-round training energy is reported per eq. (4), yielding `training_model_attributable_mJ` — the model-attributable component used in Section **??** and Figure 3 (Pareto).

$$E_{\text{round}} = E_{\text{total}} - P_{\text{idle}} \cdot \Delta t_{\text{round}} \tag{4}$$

This protocol follows the ML.ENERGY 2025 best-practices recommendation [37] of preferring the cumulative Energy API over polling-power Riemann integration. Single-GPU only (multi-GPU not handled by design); `nvidia-ml-py` is the canonical Python binding (the deprecated `pynvml` package is used only as a fallback, with a deprecation warning logged at startup).

E. Statistical inference

Statistical inference protocol (preregistered):

1) **Per-cell summary statistic.** Mean ± standard deviation of test AUC across the 10 seeds.
2) **Pairwise comparison statistic.** Paired-bootstrap CI_{95} on the per-seed AUC delta via `scripts/aggregate_v7_results.py`: `paired_bootstrap_delta` ($n_{\text{boot}} = 10\,000$, percentile interval). We choose percentile over BCa because $n = 10$ with approximately symmetric per-seed delta distributions makes the BCa bias-correction term smaller than the seed σ on every reported finding (Section **??**).
3) **Pairwise axis fix.** Each pairwise comparison holds all axes other than the contrasted one fixed (e.g., FedAdam-vs-FedAvg holds (arch, partition) constant; Spiking-vs-LSTM holds (algorithm, partition) constant).
4) **Bootstrap RNG decorrelation.** Each pairwise comparison draws an independent BLAKE2b-derived seed offset added to the base bootstrap seed. Without this, every pair would resample at exactly the same indices, producing artificially correlated CIs and breaking any joint coverage claim across the 270 paired-bootstrap distributions.
5) **Pair-set base seeds.** 2026 for the 180 algorithm-pair sweep; 2027 for the 90 architecture-pair sweep. The two sweeps' bootstrap streams are independent.
6) **Secondary check.** Wilcoxon signed-rank p-value, as a directional sanity check; the paired-bootstrap CI is the primary statistic (preregistered).
7) **Multi-comparison correction.** p-values reported uncorrected per finding. Family-wise Bonferroni at $\alpha = 0.05$ corresponds to threshold $0.05/180 \approx 2.78 \times 10^{-4}$ over the algorithm-pair family and $0.05/90 \approx 5.56 \times 10^{-4}$ over the arch-pair family. For the strictest joint coverage claim across the entire 270-pair set, the Mamba × SCAFFOLD finding (uncorrected Wilcoxon $p = 0.002$, see Section **??**) sits above the corrected threshold, but its paired-bootstrap CI_{95} excludes 0 with margin ($\Delta = -0.0915$, CI $[-0.1288, -0.0544]$). Readers seeking strict family-wise control should treat this Wilcoxon as supportive but not primary evidence.

V. Reproducibility infrastructure

We outline the artefact components shipped with the paper. Camera-ready release includes a Zenodo-archived snapshot under Apache-2.0 with Croissant metadata; this section documents the structure so reviewers can audit the shape of the release before acceptance.

A. Spec-driven YAML pipeline

Every experiment cell is fully specified by a single YAML file under `experiments/specs/` (e.g., `stage2_full.yaml` for Phase 5; `ablation_random_split.yaml` for Section **??**). The launcher (`experiments/run_v7_phase_sweep.py`) reads the spec, expands the Cartesian product (archs × algorithms × partitions × seeds), and dispatches per-cell runs with a `--skip-completed` resume mechanism that recovers from cluster pre-emption without recomputing finished cells. This eliminates the most common reproducibility friction in multi-day FL sweeps (silent re-execution of completed cells).

B. Test infrastructure

The release ships 277 passing tests across four suites (202 trainer + 22 aggregator + 37 paper invariants + 16 paper claims-vs-data); the latter two (53 tests) form a developer pre-commit gate that verifies every `PAPER_DRAFT.md` edit. Full breakdown in App. B.1.

C. Statistical pipeline

`scripts/aggregate_v7_results.py` produces `aggregated_phase5.json` (90 group means + 270 paired-bootstrap distributions); BLAKE2b-hashed independent bootstrap streams support joint coverage claims (Section IV-E). Full pipeline detail in App. B.2.

D. Croissant dataset metadata

The preprocessed `coloran_raw_unified.parquet` is documented in a Croissant 1.0 metadata file shipped with the release archive (App. B.3).

E. Reproducible execution environment

Release archive bundles `Dockerfile.repro` (CUDA 12.8 / PyTorch 2.10 pinned), `requirements.lock` (SHA256-pinned 74 packages), and `repro.sh` (1-cell smoke + bit-equivalence check). Full detail in App. B.4.

F. Demo notebook

`notebooks/colosseum_oran_federated_slicing_demo.ipynb` is the recommended onboarding entry point. See App. B.5.

References

[1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "ColO-RAN: Developing machine learning-based xApps for Open RAN closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing*, 2022.
[2] S. Caldas, S. M. K. Duddu, P. Wu, T. Li, J. Konečný, H. B. McMahan, V. Smith, and A. Talwalkar, "LEAF: A benchmark for federated settings," *arXiv:1812.01097*, 2018.
[3] F. Lai, Y. Dai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "FedScale: Benchmarking model and system performance of federated learning at scale," in *ICML*, 2022.
[4] o. Song, "FLAIR: Federated learning annotated image repository," 2022.
[5] F. Granqvist *et al.*, "pfl-research: Simulation framework for accelerating federated learning research," in *NeurIPS Datasets and Benchmarks*, 2024, arXiv:2404.06430.

[6] o. Shen, "STEP: A unified spiking transformer evaluation platform for fair and reproducible benchmarking," *arXiv:2505.11151*, 2025.

[7] (authors), "Fed-LSTM: LSTM forecaster for slice-level traffic in cellular FL," 2022.

[8] ——, "FedAvg/FedProx/FedBN benchmark on five throughput-prediction datasets," 2025, arXiv:2508.08479.

[9] Mangi *et al.*, "When Clients Drift: Regime-aware aggregation in 6G RAN telemetry," 2026.

[10] L. Bonati *et al.*, "Colosseum: Large-scale wireless experimentation through hardware-in-the-loop network emulation," *IEEE Open Journal of the Communications Society*, Aug. 2024.

[11] J. Liu *et al.*, "FedSWA: Federated stochastic weight averaging," in *ICML*, 2025, arXiv:2507.20016.

[12] "FedSCAM: Sharpness-aware clustering for federated learning," 2026, arXiv:2601.00853 (Jan 2026).

[13] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "FedBN: Federated learning on non-IID features via local batch normalization," in *ICLR*, 2021, arXiv:2102.07623.

[14] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," 2020, arXiv:2007.14390.

[15] (authors), "fev-bench: Forecast evaluation with bootstrap-ci for time-series forecasting benchmarks," 2025.

[16] ——, "Statistical federated learning for beyond-5g SLA-constrained RAN slicing," *IEEE*, 2021.

[17] ——, "CDF-aware federated learning for RAN slicing under long-term SLA constraints," *IEEE*, 2021.

[18] J. Groen *et al.*, "TRACTOR: Traffic analysis and classification tool for open RAN," in *IEEE*, 2023.

[19] M. Barker *et al.*, "REAL: An open-source reinforcement learning framework for the near-rt RIC," 2025, arXiv:2502.00715.

[20] Hayek *et al.*, "Empirical evaluation of federated learning convergence over heterogeneous COTS networks," 2025.

[21] Chen *et al.*, "SpikMamba: When SNN meets Mamba for event-based action recognition," in *(venue TBD)*, 2024.

[22] (authors), "Mamba-Spike: Hybrid spiking front-end with selective state-space backbone for temporal data," 2024, arXiv:2408.11823.

[23] ——, "Spiking point Mamba for 3D point cloud processing," in *ICCV*, 2025.

[24] ——, "SpikingSSMs: Sparse-parallel spiking state-space models," in *AAAI*, 2025.

[25] ——, "SpikingMamba: Distilling large language models into spiking-SSM variants for energy-efficient inference," 2025, arXiv:2510.04595.

[26] ——, "SpikingBrain: Scaling spiking architectures to 76B parameters with explicit FLOP-utilization measurement," 2025, arXiv:2509.05276.

[27] ——, "Federated learning with spiking neural networks," 2021, arXiv:2106.06579.

[28] ——, "FedLEC: Federated learning with spiking neural networks under label-skew non-iid," 2024, arXiv:2412.17305.

[29] ——, "Spiking neural networks in vertical federated learning," 2024, arXiv:2407.17672.

[30] ——, "Robustness of spiking neural networks in federated learning with compression," 2025, arXiv:2501.03306.

[31] ——, "Privacy in federated learning with spiking neural networks," 2025, arXiv:2511.21181.

[32] ——, "Firing-rate sensitivity to differential privacy in federated spiking neural networks," 2026, arXiv:2602.12009 (Feb 2026).

[33] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *COLM*, 2024, arXiv:2312.00752.

[34] Shen *et al.*, "Is conventional SNN really efficient? a perspective from bit budget," *(venue TBD)*, 2023.

[35] A. Asperti *et al.*, "α-FLOPs: A parallelism-aware alternative to flop counting for energy estimation," *(venue TBD)*, 2021.

[36] J. Yik *et al.*, "NeuroBench: A framework for benchmarking neuromorphic computing algorithms and systems," *Nature Communications*, 2025.

[37] J.-W. Chung *et al.*, "The ML.ENERGY benchmark: Measuring inference energy of generative AI models," 2025, arXiv:2505.06371.

[38] ——, "Where do the joules go? diagnosing energy breakdowns across generative-model families," 2026.

[39] Spyra *et al.*, "Beyond backpropagation: NVML-instrumented energy profiling of alternative training objectives," 2025.

[40] Q. Li, Y. Diao, Q. Chen, and B. He, "NIID-Bench: Federated learning on non-IID data silos: An experimental study," in *ICDE*, 2022.

[41] (authors), "pFedFDA: Personalized federated learning under covariate shift via feature-distribution alignment," in *NeurIPS*, 2024.

[42] Borazjani *et al.*, "Embedding-based heterogeneity definitions for federated learning beyond label-skew," 2025, arXiv:2503.14553.

[43] D. Caldarola, B. Caputo, and M. Ciccone, "Improving generalization in federated learning by seeking flat minima (FedSAM)," in *ECCV*, 2022.

[44] (authors), "atan surrogate gradient for spiking neural networks (wei et al. 2018)," 2018, citation key kept verbatim from PAPER_DRAFT.md; venue + author list TBD at camera-ready.

[45] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[46] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks (FedProx)," in *MLSys*, 2020.

[47] S. J. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, "Adaptive federated optimization (FedAdam)," in *ICLR*, 2021.

[48] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *ICML*, 2020.

[49] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization (FedDyn)," in *ICLR*, 2021.

[50] Q. Li, B. He, and D. Song, "Model-contrastive federated learning (MOON)," in *CVPR*, 2021.

[51] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," in *NeurIPS Workshop on Federated Learning*, 2019, arXiv:1909.06335.